

K Nearest Neighbours (KNN)

What is KNN?

- K-Nearest Neighbors (KNN) is a supervised learning algorithm.
- It can be used for both classification and regression.
- It’s based on the idea of similarity: similar data points are near each other.
- KNN is a Lazy Learning Algorithm (vs Eager Learning algorithms we’ve seen before)
- KNN is a Non-Parametric Method (does not assume a specific form or structure for the function that maps input variables to an output variable)

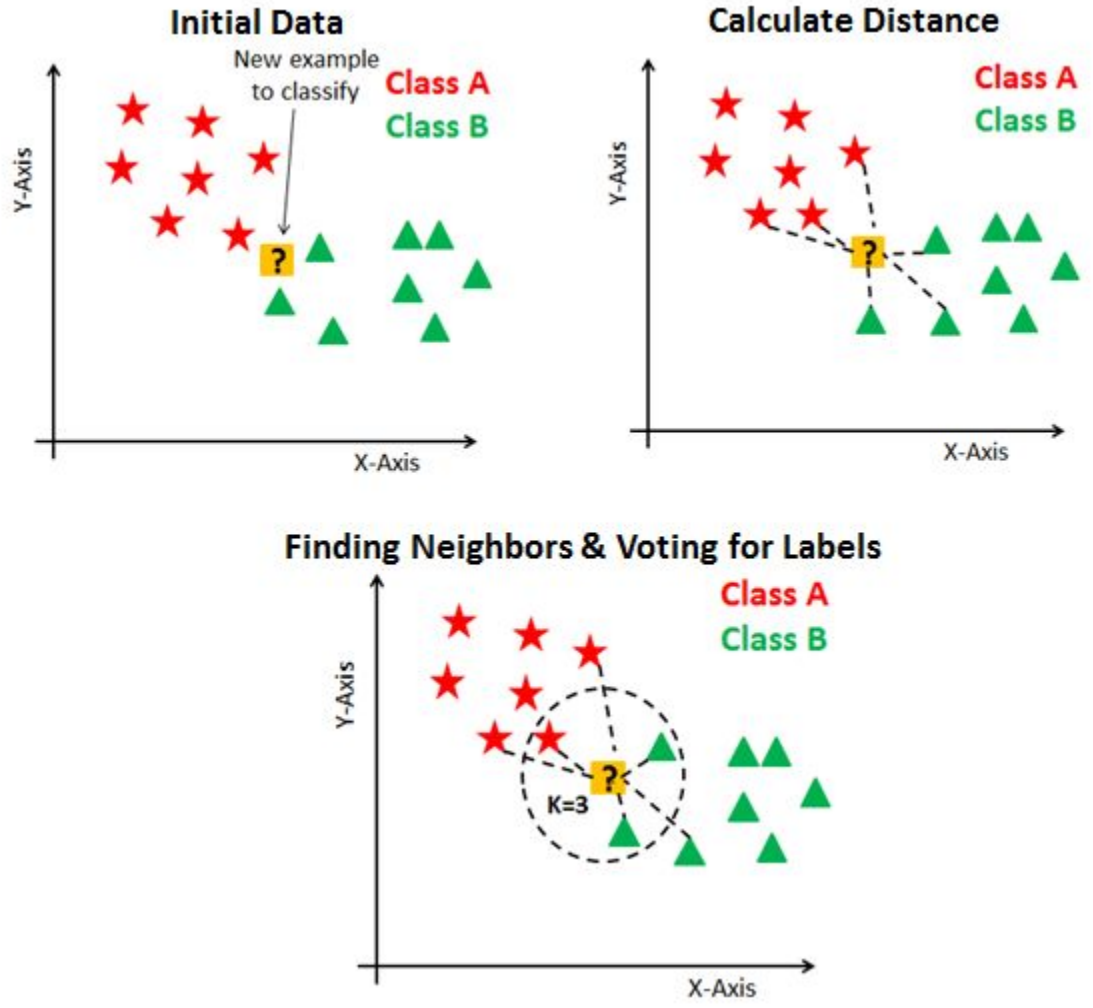
Key Idea:
To predict a label for a new data point, look at the "k" closest known data points and use a majority vote (classification) or average (regression).

How KNN Works

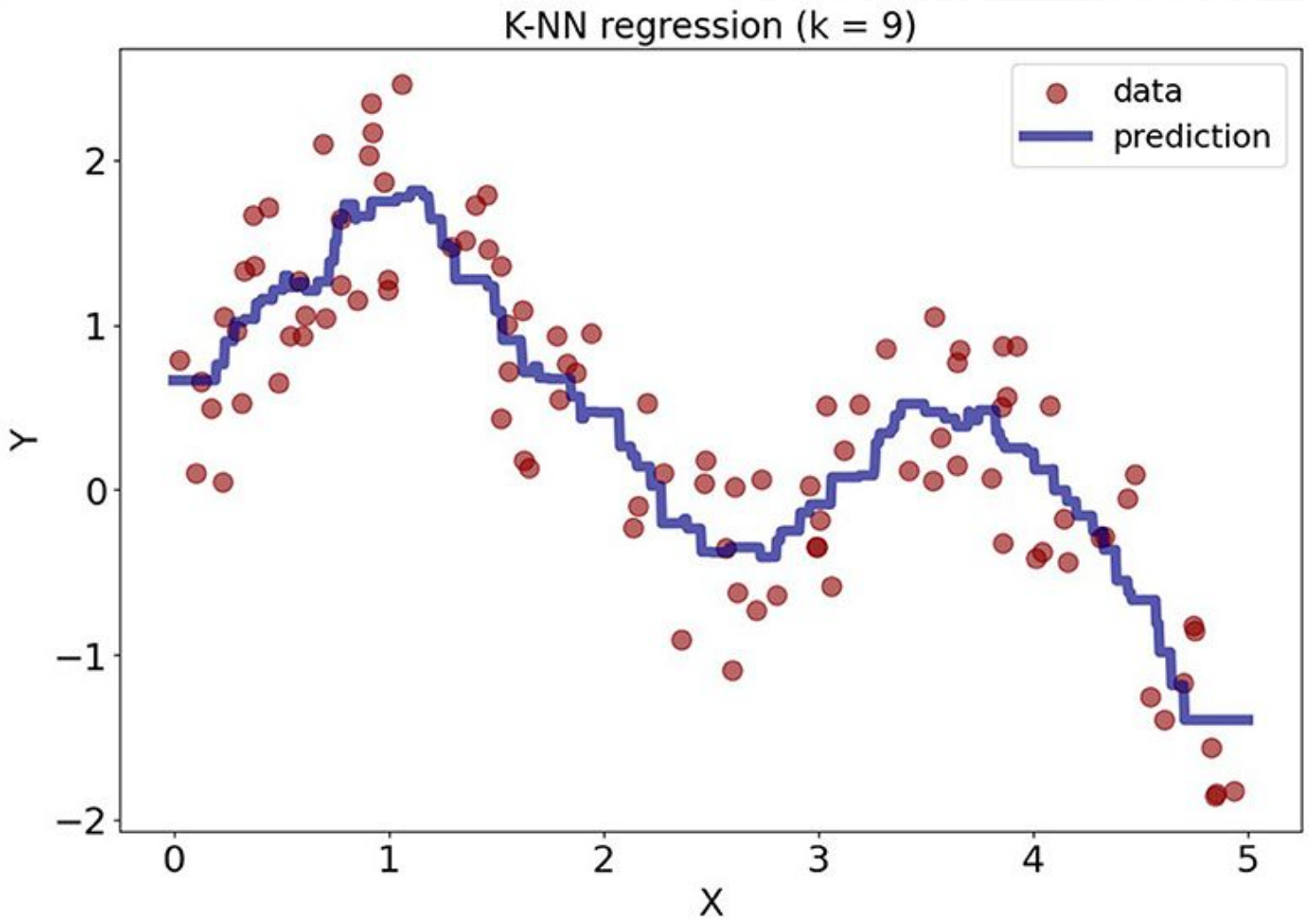
1. Choose k (e.g., 3, 5, 7)
2. Calculate distance (usually Euclidean) between the new point and all existing points
3. Find the k closest neighbors
4. Predict:
    a. Classification: Most common label among neighbors
    b. Regression: Average value of neighbors

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

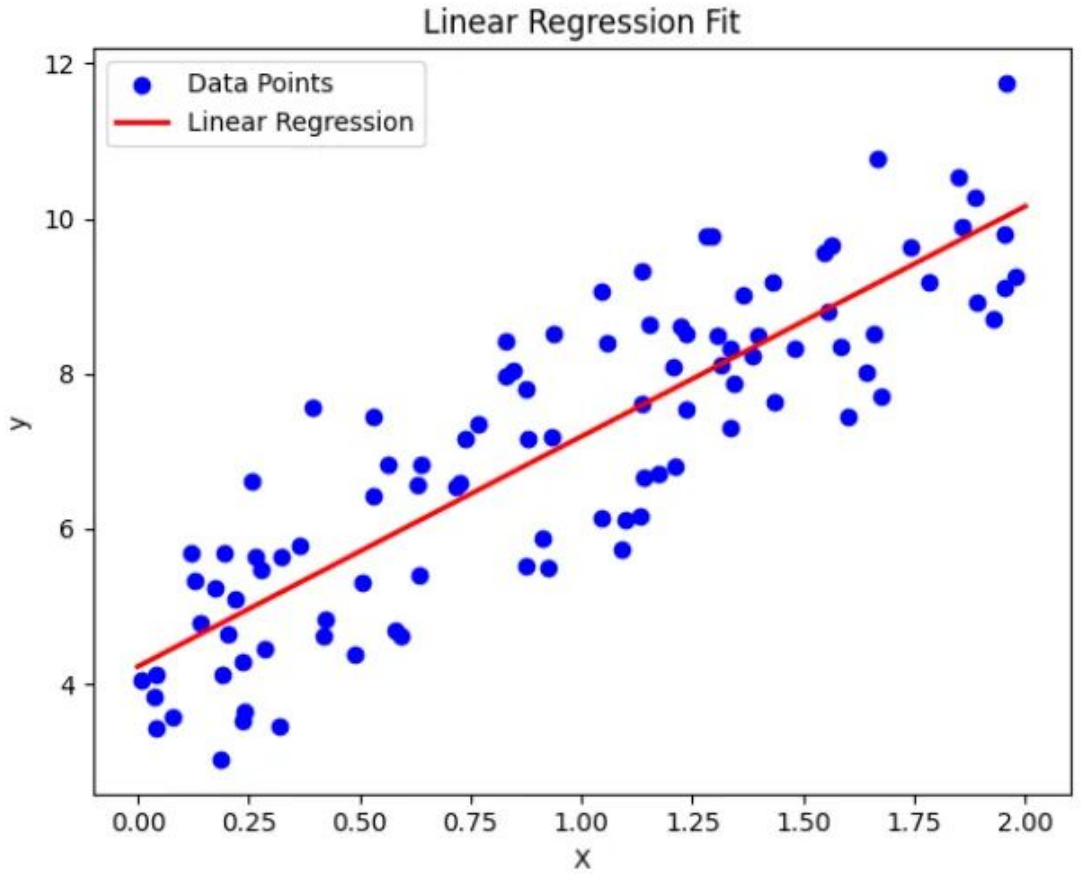
KNN - Classification visualization

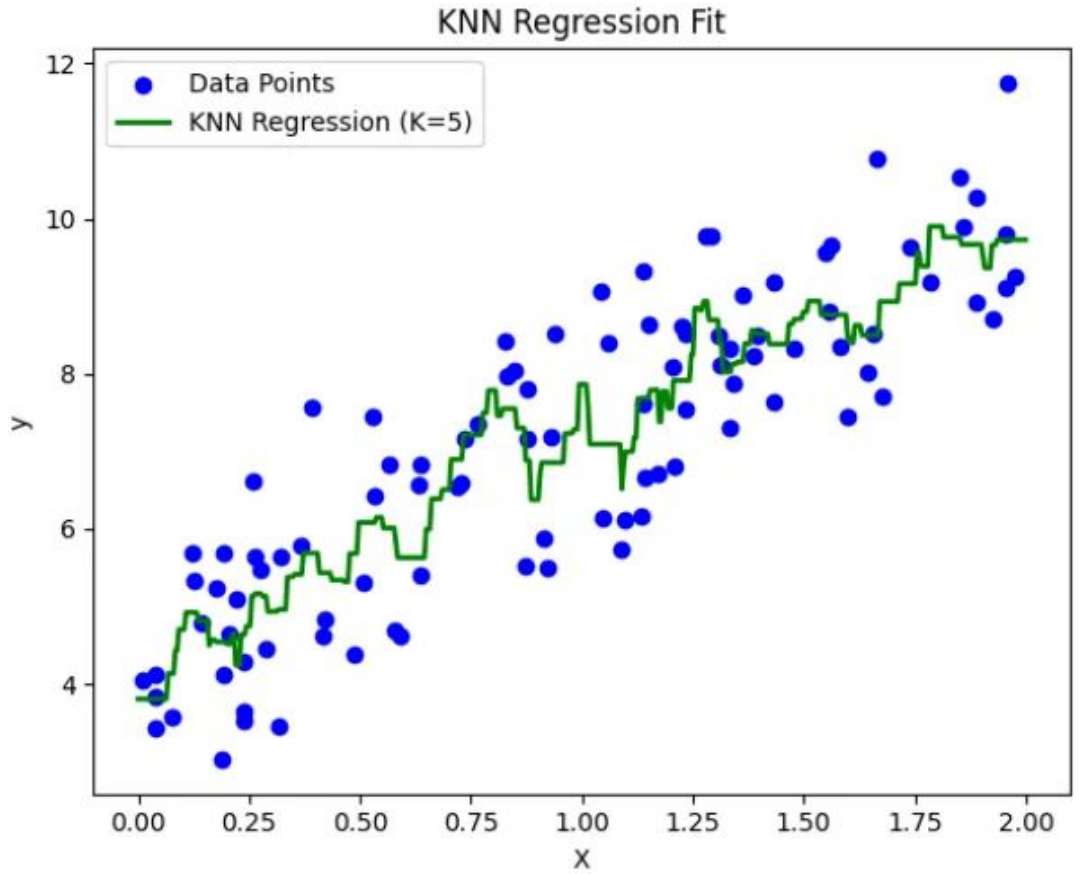


KNN - Regression visualization



Linear Regression vs KNN Regression Fit





JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

Eager Learning vs Lazy Learning

Eager Learning

- Learns a model during training
- Generalizes before seeing the test data
- Fast prediction, but training may be slow
- Examples: Logistic Regression, Decision Trees, Neural Networks, SVM

Lazy Learning

- Defers learning until a prediction is needed
- No model is built upfront
- Fast training, but slow predictions
- Examples: K-Nearest Neighbors (KNN)

Parametric vs Non-Parametric methods

Parametric

- Assumes a specific form for the model (e.g., linear)
- Fixed number of parameters
- Efficient, but less flexible
- Examples: Linear Regression, Logistic Regression, Naive Bayes

Non-Parametric

- No fixed structure; model grows with data
- Can capture complex patterns
- More flexible, but needs more data and compute
- Examples: KNN, Decision Trees, Random Forests, SVM with kernels

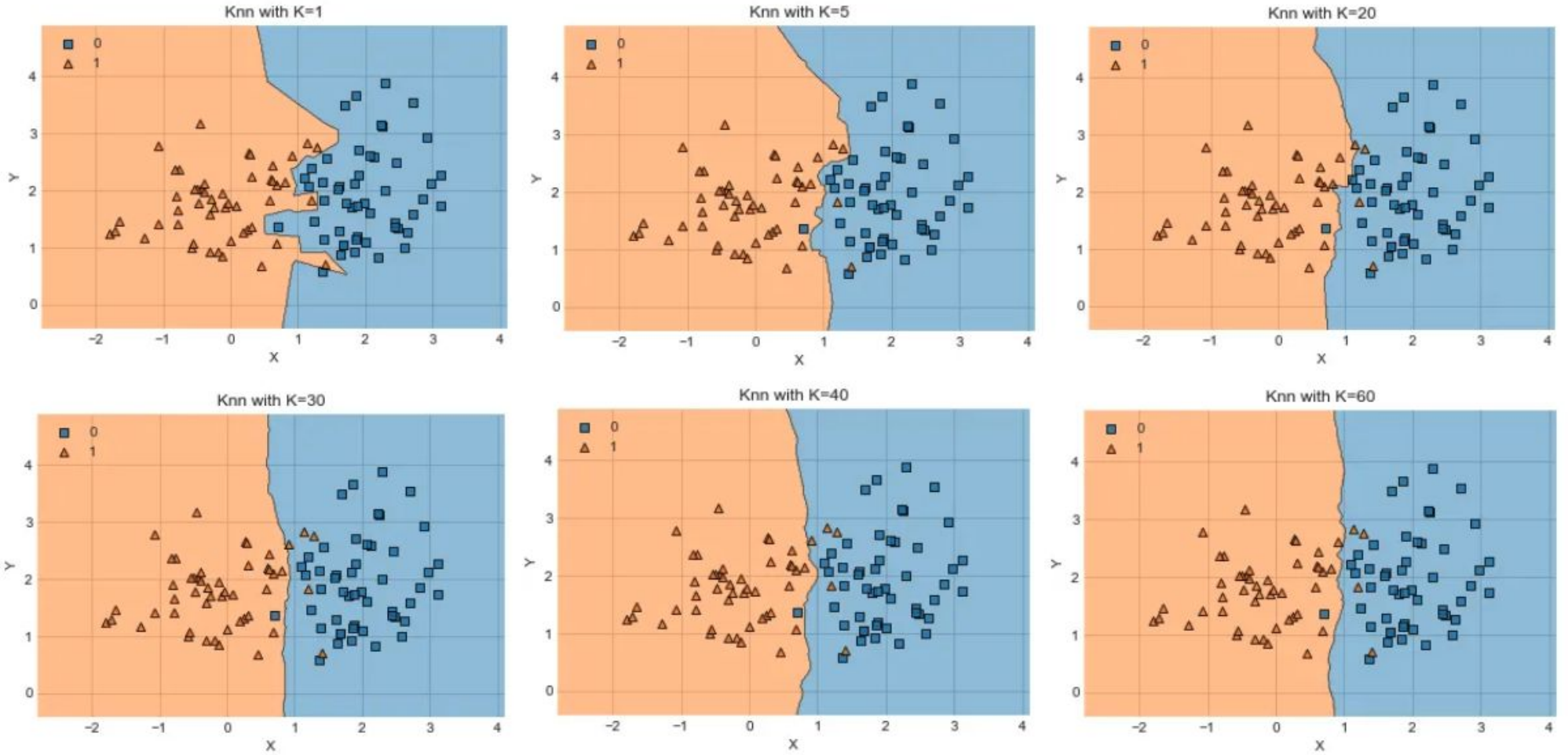
JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

Choosing the Right Value of K

- Small k (e.g. 1): Can lead to overfitting
- Large k: Smoother decision boundary but may ignore local structure
- Typically, odd values are used to avoid ties (for classification)
- Use cross-validation to find the optimal k.

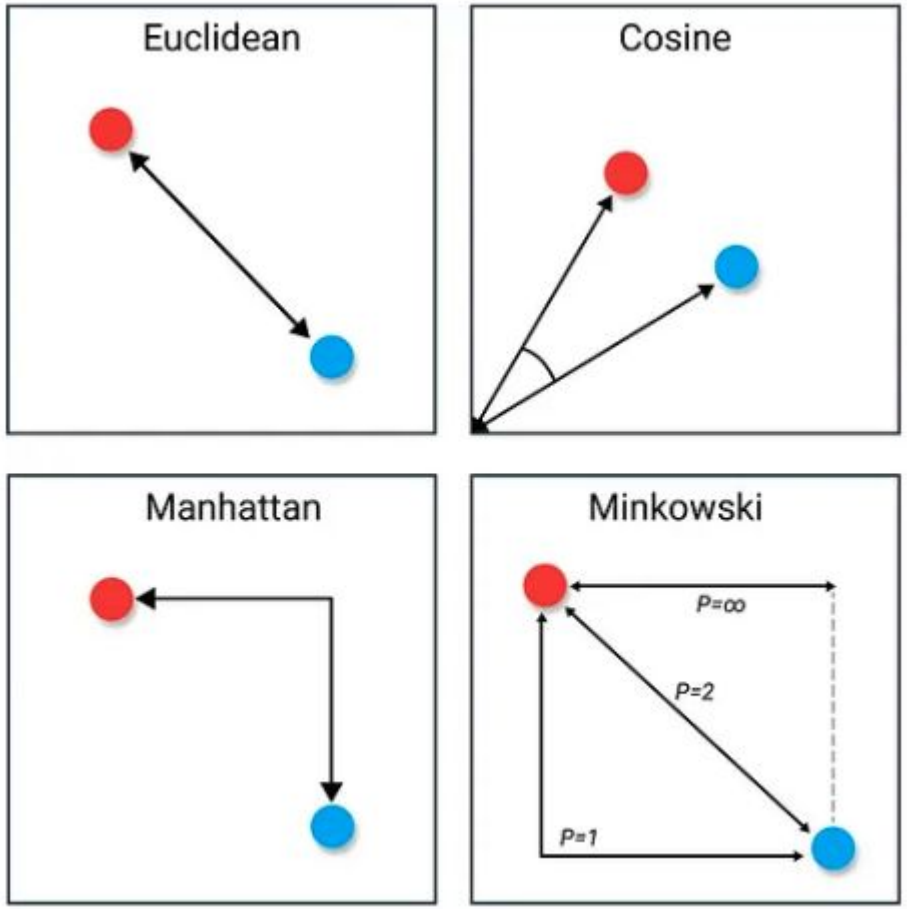
JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

Selecting K visualization



KNN visualization for the linearly separable dataset

Distance Metrics



JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

Distance Metrics

- **Euclidean distance (default)**: suitable for data that has continuous and numerical features with similar scales and ranges. It can also handle outliers and noise well, as it gives more weight to larger differences. However, it can be affected by the curse of dimensionality, which means that as the number of features increases, the distance between any two points becomes less meaningful and more similar.
- **Manhattan distance**: suitable for data that has discrete and categorical features, as it does not penalize small differences as much as the Euclidean distance. It can also handle high-dimensional data better, as it is less sensitive to the curse of dimensionality. However, it can be influenced by the orientation and scale of the features, as it assumes that all directions are equally important and all units are comparable.

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

Distance Metrics

- **Minkowski distance**: generalization of the Euclidean and Manhattan distances. It is suitable for data that has mixed types of features, as it allows you to adjust the parameter ***p*** to balance the importance of different features and distances.
- **Cosine similarity**: the cosine of the angle between two vectors. The cosine similarity is suitable for data that has sparse and high-dimensional features, such as text or image data, as it measures the similarity based on the direction and not the magnitude of the vectors.

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

Note on Minkowski distance and parameter *p*

The Minkowski distance of order p (where p is an integer) between two points

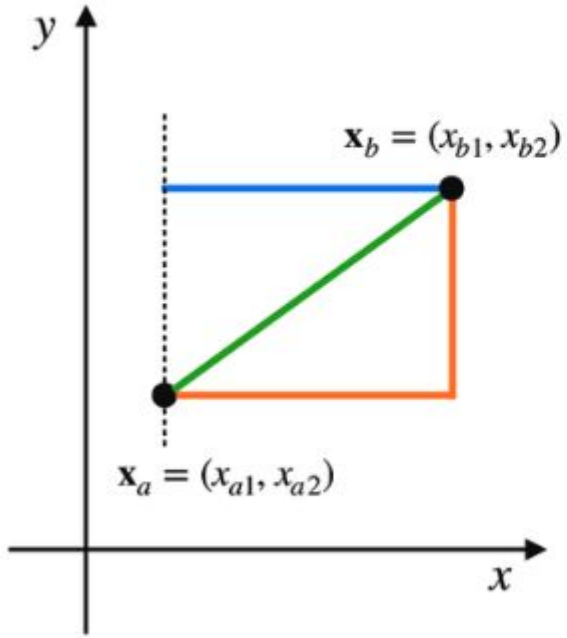
$$X = (x_1, x_2, \ldots, x_n) \text{ and } Y = (y_1, y_2, \ldots, y_n) \in \mathbb{R}^n$$

is defined as:

$$D(X, Y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{\frac{1}{p}}.$$

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

Note on Minkowski distance and parameter *p*



$p = 2$ Euclidean distance

$$\|\mathbf{x}_a - \mathbf{x}_b\|_2 = (|x_{a1} - x_{b1}|^2 + |x_{a2} - x_{b2}|^2)^{\frac{1}{2}}$$

$p = 1$ Manhattan distance

$$\|\mathbf{x}_a - \mathbf{x}_b\|_M = |x_{a1} - x_{b1}| + |x_{a2} - x_{b2}|$$

$p = \infty$ Chebyshev distance

$$\|\mathbf{x}_a - \mathbf{x}_b\|_\infty = \max\{|x_{a1} - x_{b1}|, |x_{a2} - x_{b2}|\}$$

Strengths of KNN

- **Simplicity**: KNN is easy to understand and implement. It's intuitive and doesn't require assumptions about the data's shape.
- **Lazy learning** - no training phase
- **Adaptability**: Since KNN doesn't assume a specific relationship between the features and the target variable, it can model complex data without requiring a lot of tuning.
- **Interpretability**: Predictions can be explained by simply looking at the neighbors used to make the prediction, providing a clear view of how the model arrived at a particular outcome.

Weaknesses of KNN

- **Computational Cost**: For each new prediction, KNN needs to search through the entire dataset to find the K closest neighbors, which can be slow if the dataset is large. Techniques like KD-Trees or Ball Trees can help speed up this process but can still be resource-intensive.
- **Sensitive to Irrelevant Features**: KNN considers all features equally when calculating distance. If the dataset includes irrelevant or less important features, they can mislead the model. For example, if some features have a large scale, they can dominate the distance metric. Scaling the features (like with Min-Max Scaling or Standardization) can help.
- **Sensitive to the Curse of Dimensionality**: As the number of features increases, the distance between points becomes less meaningful, and more data points may be needed for KNN to work well. This problem is known as the curse of dimensionality. Techniques like dimensionality reduction can help make KNN more effective in high-dimensional

KNN is best suited for cases where

- The dataset is relatively small or can be efficiently searched.
- The relationship between features and the target variable is complex or unknown, making non-parametric methods preferable.
- The data is reasonably well-scaled and doesn’t contain too many irrelevant features.
- The problem is suitable for a local approach, meaning you want to make predictions based on similar instances rather than fitting a global model.

Ways to enhance KNN's performance

- **Scaling the Data**: Scaling features to a similar range can help improve the accuracy of the distance metric, especially when features have different units.
- **Feature Selection**: Removing irrelevant or noisy features can make KNN more effective, reducing computation time and improving accuracy.
- **Weighted KNN**: Instead of simply averaging the target values of the neighbors, you can weigh each neighbor based on its distance. Closer neighbors have a larger influence, which can improve prediction accuracy.
- **Using Cross-Validation for K Selection**: Using techniques like cross-validation can help you find the best value for K by testing different values on subsets of your data and selecting the one with the lowest error.



KNN as Imputer

Using KNN for Missing Value Imputation

- Instead of using mean/median to fill missing values, KNN imputation fills them using the values from the nearest neighbors.

- For each sample with missing data:
  - Find K most similar instances (based on non-missing features)
  - Impute missing values using the average (or mode) of those neighbors

- Why KNN Imputation:
  - More accurate than mean/median
  - Preserves relationships between features
  - Suitable for both numerical and categorical data

How It Works – Step-by-Step

1. Normalize all features (important for distance computation)
2. For each missing entry:
    a. Identify K nearest rows (samples) using available values
    b. Use neighbors to estimate the missing value:
        i. Numerical: Mean or weighted average
        ii. Categorical: Mode (most frequent class)



KNN as Anomaly Detector

KNN for Outlier Detection

Core Idea:

- Outliers are data points that are far away from their neighbors.
- KNN measures the distance to the K nearest neighbors
- If a point is far from its neighbors ⟶ it may be an anomaly

How It Works

1. For each data point:
   a. Compute distances to its K nearest neighbors
   b. Aggregate those distances:
      - Mean distance
      - Max distance
      - Sum of distances

2. Rank data points by distance score
   a. High distance = less similar to other points = potential outlier

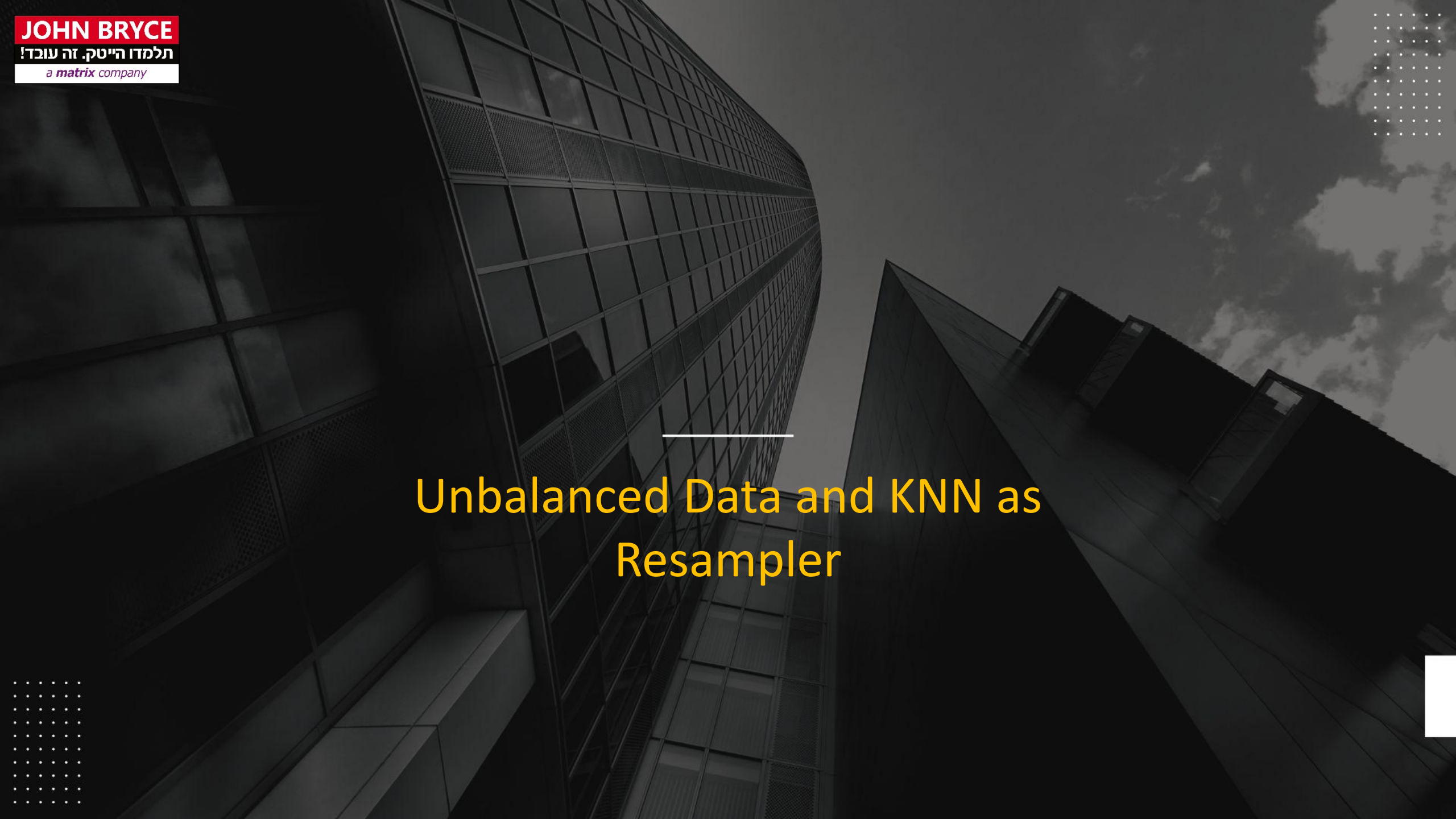
Pros and Cons

✅ Advantages

- Non-parametric (no assumptions about data distribution)
- Works for unsupervised anomaly detection
- Easy to interpret

❌ Limitations

- Sensitive to feature scaling
- Expensive on large datasets
- Choice of k and threshold can be tricky



Unbalanced Data and KNN as Resampler

Understanding Unbalanced Data

What is Unbalanced Data?

- Occurs when classes in the target variable are not represented equally
- Common in classification problems (e.g., fraud detection, rare disease diagnosis)

Why is it a Problem?

- Algorithms like LogisticRegression assume balanced class distribution
- Model may be biased toward the majority class
- Metrics like accuracy can be misleading

Tackling Unbalanced Data

1. Use Weighted Learning
    - Class weights (classification - use class weights to balance impact of minority class)
    - Sample weights (regression - use sample weights if some samples should influence the loss more)
2. Use Resampling (classification)
    - Oversample minority class
    - Undersample majority class

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

Weighted Learning

- In weighted learning, each training sample is given a weight that reflects its importance in the loss calculation.
- Instead of minimizing the average loss, the algorithm minimizes the weighted average loss:

$$\text{Loss}_{\text{weighted}} = \frac{1}{\sum w_i} \sum_{i=1}^{N} w_i \cdot \text{Loss}(y_i, \hat{y}_i)$$

- During training, for each sample in the loss function:
  - Samples from minority classes get higher weights → their misclassifications "hurt" more
  - Samples from majority classes get lower weights
- This affects gradient updates — **samples with higher weights contribute more to parameter updates**.

Undersampling – Reduce the Majority Class

- **Undersampling is a technique to balance the dataset by reducing the number of majority class samples.**
  - Keeps all minority class samples
  - Randomly removes majority samples
  - Makes class distribution more balanced

- **When to Use It**
  - When you have lots of data
  - When you want faster training
  - When minority class is small and can’t be oversampled safely

- **Drawbacks**
  - Can discard valuable data from the majority class
  - Risk of underfitting (especially with small datasets)
  - Doesn’t generate new information — just removes data

Oversampling – Boost the Minority Class

- Oversampling is a technique to balance class distribution by adding more samples to the minority class
- Helps models learn minority patterns
- Prevents bias toward majority class
- Works well when you can’t afford to drop majority data
- Example methods:
  - Random Oversampling: duplicates existing minority samples
  - SMOTE (SMOTEN, SMOTENC): creates synthetic samples by interpolation
  - ADASYN: like SMOTE, but focuses on hard-to-learn samples

SMOTE - KNN as Oversampler

- SMOTE stands for Synthetic Minority Oversampling Technique.
- It's a popular method for balancing imbalanced datasets by creating synthetic examples of the minority class rather than duplicating existing ones.

- How SMOTE Works:
  - Pick a sample from the minority class
  - Find its k-nearest neighbors (usually k=5) in the same class
  - Randomly pick one of those neighbors
  - Create a synthetic sample along the line between the original sample and the neighbor
  - This generates new, similar-but-not-identical samples → more diversity, less overfitting than plain duplication.

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

SMOTENC – Oversampling for Mixed Data

- SMOTENC = SMOTE for Nominal and Continuous features
- It extends SMOTE to work with datasets that contain both numeric and categorical features.
- How it Works:

Feature Type	Method Used
Numerical	Interpolation between neighbors
Categorical	**Majority voting** among neighbors

ADASYN - Adaptive Synthetic Sampling

- It's an advanced oversampling technique that builds on SMOTE, but focuses on hard-to-learn samples in the minority class
- How It Works:
  - Identify difficult samples in the minority class (based on nearest neighbors)
  - For each minority class sample, finds its k-nearest neighbors (typically k=5)
  - Check how many of those neighbors are from the majority class.
  - The more majority neighbors there are, the more likely that point is close to the decision boundary or in a difficult area.
  - Generate more synthetic samples for those difficult regions

  → Minority areas near the decision boundary are sampled more heavily

ADASYN - Pros and Cons

Pros:

- Focuses model learning where it matters most
- Can improve decision boundaries compared to uniform oversampling
- Especially helpful when the minority class is spread out or noisy

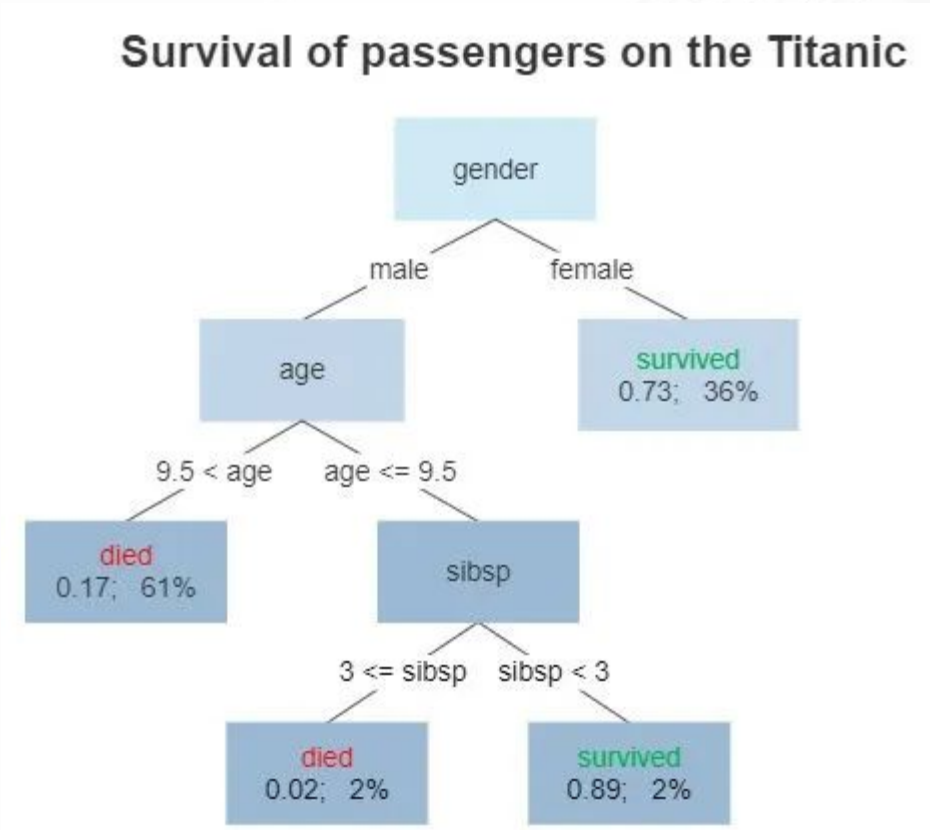
Cons:

- May generate samples near noisy or overlapping regions
- Like SMOTE, it requires numerical features (or encoded categoricals)
- Not ideal if you want balanced oversampling everywhere



Decision Trees

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company



JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

🌳 Decision Tree – Algorithm Overview & Properties

- **Supervised** Learning
- **Eager** Learner - builds the model during training, makes predictions instantly during inference
- **Non-Parametric** model - doesn’t assume a fixed number of parameters, can adapt its structure to the data complexity
- Suitable for **Classification** and **Regression** tasks
- **Interpretable**
- **Greedy** Algorithm
- **No feature scaling** needed

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

🌳 Step-by-Step: How a Decision Tree Is Built

- Step 1: Start with the Full Dataset
  - The tree starts with all training samples at the root.
  - The goal is to split the data to make the target values in each group as "pure" as possible.
    - Regression → Predict continuous values
    - Classification → Predict class labels

- Step 2: Try All Possible Splits
  - For each feature:
    - If numerical: try all possible midpoints between sorted unique values
    - If categorical: try all valid partitions (or use one-vs-rest strategy)
  - For each candidate split, divide the data into two groups.

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

🌳 Step-by-Step: How a Decision Tree Is Built

- Step 3: Evaluate Each Split
  - Calculate the impurity or error after the split:

Task	Split Quality Measure
Classification	Gini impurity or Entropy
Regression	Mean Squared Error (MSE) or Mean Absolute Error (MAE)

- Compute the weighted sum of impurity/errors from both child nodes.

🌳 Step-by-Step: How a Decision Tree Is Built

- Step 4: Choose the Best Split
  - Select the feature and threshold that produce the lowest total impurity.
  - This is a greedy decision: pick the best option at this level only.

- Step 5: Repeat Recursively
  - Apply the same splitting process to each child node.
  - Stop when one of the following is true:
    - Maximum depth is reached
    - Minimum number of samples per node is too small
    - Node is pure (classification) or variance is low (regression)

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

🌳 Step-by-Step: How a Decision Tree Is Built

- Step 6: Make Predictions
  - At each leaf node, store the prediction:
    - Classification → majority class
    - Regression → mean target value

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

🌳 Decision Tree - Greedy Algorithm

- A **greedy algorithm** makes a decision that seems **best at the current step**, hoping that these **local optimal choices** will lead to a **globally optimal solution**.

- **At each step of building the tree, the algorithm**:
  - Selects the best feature and threshold that produces the best immediate split (i.e., minimizes impurity or maximizes information gain)
  - It does not look ahead or consider future splits — just chooses what looks best right now

- **Consequence of Being Greedy**:
  - Fast and efficient
  - But… might not find the globally optimal tree
  - Can lead to overfitting, especially with noisy data or many features

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

💰 Example: Coin Change Problem (Greedy Algorithm)

Goal:

- Given a set of coin denominations and a target amount, return the fewest number of coins needed to make that amount.

🇺🇸 Works Well Example (U.S. Coins):

Coins: `[1, 5, 10, 25]`

Target: `30`

Greedy picks:

- 25 → remaining = 5
- 5 → remaining = 0

  ✅ Total coins = **2** → this *is* optimal

❌ Fails Example:

Coins: `[1, 3, 4]`

Target: `6`

Greedy Algorithm:

- Pick 4 → remaining = 2
- Pick 1 → remaining = 1
- Pick 1 → remaining = 0

  🔴 Total coins = 3

Optimal Solution:

- 3 + 3

  ✅ Only **2** coins!

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

🌳 How Feature Importance is Calculated in Decision Trees

- **Concept**: A feature's importance is based on how much it **reduces impurity** across all the splits it appears in — summed up over the whole tree.
- Calculation Steps:
  - Every time a node is split, the model calculates the decrease in impurity (e.g., Gini, Entropy, or MSE).
  - This decrease is attributed to the feature used for that split.
  - The importance for each feature is the sum of all decreases in impurity it contributed across the tree.
  - Finally, all importances are normalized so they sum to 1.

$$\text{Importance}(f) = \sum_{\text{splits on } f} [\text{Impurity(parent)} - (\text{Weighted impurity of children})]$$

Tree pruning

- **Tree pruning** is the process of **reducing the size of a decision tree** by removing nodes that provide little to no value.
- Fully grown trees tend to **overfit** the training data
- Pruning improves **generalization** to new/unseen data
- **Types of Pruning**:
  - **Pre-Pruning** (Early Stopping) - stop growing the tree early based on constraints (e.g., max_depth, min_samples_split, min_impurity_decrease)
  - **Post-Pruning** (Reduced Error or Cost Complexity) - grow the full tree, then prune back branches that don’t improve validation accuracy or penalize complexity

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

Pre-Pruning

- Maximum depth of the tree (max_depth): Limits the depth of the tree.
- Minimum samples per leaf (min_samples_leaf): Ensures a minimum number of samples in each leaf.
- Minimum samples to split (min_samples_split): Requires a minimum number of samples to perform a split.
- Maximum number of nodes (max_leaf_nodes): Limits the number of leaf nodes in the tree.
- Others (see sklearn documentation)

Post-Pruning

- ccp_alpha – Cost Complexity Pruning
- Complexity Parameter (α) used for Minimal Cost-Complexity Pruning
  - After growing the full tree, scikit-learn can prune branches that offer minimal gain but add complexity.
  - It removes branches where the cost-complexity tradeoff is not favorable.
  - As alpha increases, more of the tree is pruned, which increases the total impurity of its leaves.
- Visualizations (from sklearn)

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

🌳 Evaluation for Classification - Gini Impurity

🔷 **Gini Impurity Formula:**

$$Gini = 1 - \sum_{i=1}^{n} p_i^2$$

Where:

- p_i = proportion of class i in the node
- The sum runs over all classes in the node

🔍 **Examples:**

- Pure node: [Yes, Yes, Yes]
  → Gini = $1 - 1^2 = 0$
- Mixed node: [Yes, No]
  → $p_{Yes} = 0.5,\ p_{No} = 0.5$
  → Gini = $1 - 0.5^2 - 0.5^2 = 0.5$

- Gini impurity measures how often a randomly chosen sample would be incorrectly labeled if it was randomly labeled according to the distribution of labels in the node.
- Low Gini ⟶ Pure node (mostly one class)
- High Gini ⟶ Impure node (mixed classes)

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

🌳 Evaluation for Classification - Entropy

🔷 **Entropy Formula:**

$$Entropy = -\sum_{i=1}^{n} p_i \cdot \log_2(p_i)$$

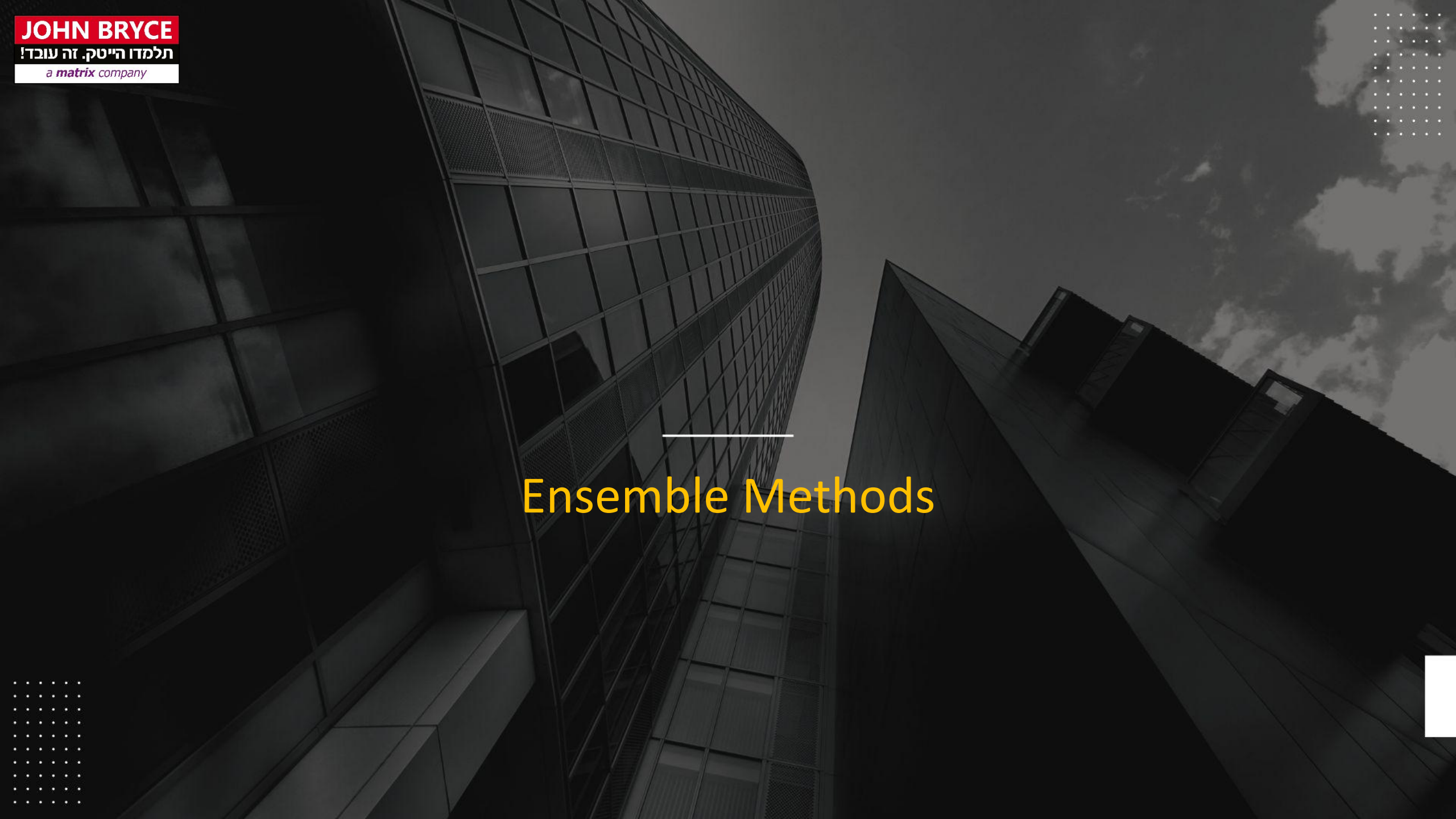
Where:

- p_i is the proportion of class i in the node
- The sum runs over all classes

📊 **Examples:**

Class Distribution	Entropy
[Yes, Yes, Yes]	0.0
[Yes, No]	1.0
[Yes, Yes, No]	~0.918

- Entropy measures impurity or disorder in a node
- Think of it as "how mixed" the classes are
- Higher entropy ⟶ classes are more evenly distributed (more uncertainty)
- Lower entropy ⟶ one class dominates (more certainty)



Ensemble Methods

Ensemble methods - Why Combine Models?

- **Ensemble methods** combine predictions from **multiple models** to produce a stronger overall model.
- Instead of relying on a single model, we **build a "team" of models** that work together.
- **Core Idea** - different models make different mistakes, together, they can cover each other's weaknesses.
- Why Do We Need Ensemble Methods?

Problem with Single Models	**How Ensembles Help**
May overfit or underfit	Combine multiple models to balance it
Sensitive to noise	Reduces variance via averaging
Limited accuracy	Boosts performance through diversity

Key Types of Ensembles

- **Bagging** – parallel learning, reduces variance
- **Boosting** – sequential learning, reduces bias
- **Voting** – average or majority of different models
- **Stacking** – trains a model to combine others

Bagging (Bootstrap Aggregating)

- 🎯 **Goal**: reduce variance and prevent overfitting by training multiple models on different random subsets of the data.
- 🔁 **How It Works**:
  - Draw bootstrap samples (with replacement) from the training data.
  - Train a model on each sample (e.g., decision trees).
  - Combine predictions:
    - Classification → majority vote
    - Regression → average
- 👍 **Best For**:
  - High-variance models like deep decision trees
  - Noisy datasets
- 🌲**Example**: Random Forest

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

🌲 Random Forest – A Powerful Bagging Ensemble

🔁 **How It Works**:

- Take bootstrap samples from the training data.
- For each tree:
  - Train on a different bootstrap sample.
  - At each node split, select a random subset of features (not all features).
- Combine predictions:
  - Classification ⟶ majority vote
  - Regression ⟶ average

Why Tree Pruning Isn't Usually Needed in Random Forest

- Each tree in the forest is intentionally overfit
- Random Forests usually grow fully expanded trees (until all leaves are pure or contain minimum samples).
- That’s okay — because:
  - Each tree sees different data (bootstrap sampling),
  - Each tree uses random subsets of features, and
  - The final prediction is based on averaging or majority voting across trees.

- **This ensemble effect reduces overfitting, even if individual trees are overfit.**

- Practical Advice
  - Try default settings first (unpruned, deep trees)
  - Use cross-validation to tune max_depth or min_samples_leaf only if needed.
  - For big data or production use, controlling depth helps performance.

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

Common Random Forest hyperparameters

- n_estimators: number of trees (100–300 is common).
- max_features='sqrt': standard for classification tasks.
- max_depth=None: trees grow until all leaves are pure or reach min samples.

Boosting – Learn from Mistakes

- **Key Idea:** boosting is an **ensemble technique** that builds a strong model by combining many **weak learners**, typically shallow decision trees. Each new model is trained to **fix the mistakes made by the previous models**.

- **Goal**: reduce both bias and variance by training models sequentially and learning from mistakes of the previous model.

- Main boosting types:
  - AdaBoost (Adaptive Boost)
  - Gradient boosting (XGBoost, LightGBM, CatBoost)

AdaBoost – Adaptive Boosting Explained

- AdaBoost is a boosting algorithm that focuses on the misclassified examples by adjusting their weights.
- How it works:
  - Start with all samples having equal weights.
  - Train a weak learner (typically a shallow decision tree).
  - Evaluate the learner's performance:
    - Increase weights on misclassified samples.
    - Decrease weights on correctly classified ones.
  - Train the next model on the updated weights.
  - Repeat for multiple rounds.
  - Final prediction is a weighted vote of all models - each weak learner (e.g., a decision stump) gets a weight based on how well it performed — i.e., its error rate on the training data.

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

Gradient Boosting – Learn from the Residuals

- Gradient Boosting is an ensemble technique that builds a strong model by adding weak models (usually shallow trees), each trained to correct the errors of the previous ones.
- It uses gradient descent to minimize the overall prediction error.
- 🔁 How It Works – Step by Step
  - Start with a simple initial model
  - Compute the residuals (errors) between prediction and actual values
  - Train a new model to predict those residuals (corrections to the initial result)
  - Add the predictions (scaled by learning rate) to the existing model

    $$F(x) = F_0(x) + \alpha_1 h_1(x) + \alpha_2 h_2(x) + \dots$$
  - Repeat steps 2–4 for many iterations.

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

Gradient Boosting: Additive Model

- Example: Predict car price (in $)
- Let's say we're predicting: y = car price in $
- We want to learn a function F(x) such that: F(x)≈y
- We construct this function **iteratively**:

$$F(x) = F_0(x) + \alpha h_1(x) + \alpha h_2(x) + \cdots + \alpha h_M(x)$$

- $F_0(x)$: our **initial guess** for the price — often just the **mean** of all prices in the training set.

- $h_m(x)$: each of these learns to predict a **correction** — something we can *add* to get closer to the true price.

- α (alpha): the learning rate — controls how much of each correction we actually apply.

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

🚗 Let’s Walk Through a Concrete Example

- Actual car price: y=24000
- Our initial guess (mean price): $F_0(x) = 20,000$
- Residual (error): $r_1 = y - F_0(x) = 4,000$
- So now we train a small regression tree h1(x) that tries to predict this **residual**:

$$h_1(x) \approx 4,000$$

- Assume it predicts: $h_1(x) = 3,800$
- We then update our prediction: $F_1(x) = F_0(x) + \alpha \cdot h_1(x)$
- Let’s say α=0.1: $F_1(x) = 20,000 + 0.1 \cdot 3,800 = 20,000 + 380 = 20,380$
- Better — but still not 24,000. So we compute a new residual:

$$r_2 = y - F_1(x) = 24,000 - 20,380 = 3,620$$

- Now we train h2(x) to predict this new residual... and so on.

🚗 Let’s Walk Through a Concrete Example

- Each residual is just "How many more dollars do we need to add (or subtract) to get closer to the true price?"
- So:
  - $F_0(x)$: $20,000
  - $h_1(x)$: add $380
  - $h_2(x)$: add $360
  - ...
  - Total $F(x)$: approaches $24,000 over time

- Each correction moves the prediction closer to the truth.
- It’s like: First guess → adjust → adjust → adjust → done ✅

Why Predict Residuals?

1. Error-Focused Learning

- By training each new model to predict the **residuals**, we force it to **focus exactly on the mistakes** made so far.
- Instead of learning the whole target from scratch again, each model only needs to learn what's **missing**.

2. Additive Modeling

In boosting (especially gradient boosting), we build a model **as a sum of many smaller models**:

$$F(x) = F_0(x) + \alpha_1 h_1(x) + \alpha_2 h_2(x) + \dots$$

- Each hi(x) is trained to predict the **residuals** of the previous model.
- This makes the final model a **refined approximation**: each learner improves upon the last.

Why Predict Residuals?

3. Gradient Descent in Function Space

Gradient Boosting can be seen as doing **gradient descent**, but instead of adjusting weights, it adjusts predictions by adding new functions.

- Residuals represent the **negative gradient** of the loss function.
- Training on residuals = **moving in the direction that reduces the loss fastest.**
- Boosting ≈ "gradient descent over functions" (not parameters).

4. Bias Reduction

- The first model might capture basic structure (e.g., linear trend).
- Residuals might still contain patterns (e.g., nonlinear effects).
- Fitting models to the residuals **adds flexibility** and helps reduce **bias**.
- Each step adds complexity only where it’s needed.

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

Bagging vs Boosting

Method	Best for Fixing	Base Model Characteristics
Bagging	**Overfitting** (High Variance)	Unstable, complex models (e.g., deep trees)
Boosting	**Underfitting** (High Bias)	Simple, weak learners (e.g., shallow trees)

❗ Real-world datasets are often somewhere in between — so:

- Try both bagging and boosting.
- Evaluate with cross-validation.
- Use interpretability and training time to guide your decision too.

Voting Ensemble Method

- 💡 **Core idea**: train multiple models (can be the same or different types), and combine their predictions

- 🔁 **How it works**:
  - Train several base models independently.
  - When making a prediction:
    - Collect predictions from all base models.
    - Combine them using either:
      - Majority vote (for classification)
      - Averaging probabilities (for soft classification)
      - Averaging outputs (for regression)

Type of voting

Type	Description
Hard Voting	Each model votes for a class → the class with most votes wins
Soft Voting	Each model outputs class probabilities → probabilities are averaged → class with highest avg probability wins

Stacking Ensemble Method

- **Goal**: train a meta-model to learn how to best combine predictions from multiple base models.
- 🔁 How It Works:
  a. Train base models (can be different types).
  b. Use their predictions as features.
  c. Train a final model (meta-learner) on these features.
- Best For: Combining models with different strengths

⚠️ Stacking - Data leakage warning!!!

- In stacking, you have:
  - Multiple base models (e.g., logistic regression, decision tree, SVM)
  - A meta-model (e.g., logistic regression) that learns how to combine the predictions of the base models
- ❗ If you train your base models on the entire training set and use those predictions as input to the meta-model — you are have data leakage problem!

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

⚠️ Stacking - Data leakage warning!!!

- Why can’t we train meta model on the same train data like base models?

- Let’s say, we trained base models:
- X_train, y_train => Trained Base Models

- Now we want to train meta model. The features input to meta-model should be predictions from base models. Let’s say we use:
- X_train => Base models => (very biased) Predictions => (very biased) Feature input to meta-model => Data Leakage!!!

Stacking - Data leakage Solution: Out-of-Fold Predictions

🔁 How It Works:

1. Split the training data into **k folds** (like in k-fold cross-validation)
2. For each fold:
   a. Train each base model on the other k−1 folds
   b. Use the trained model to predict on the **held-out fold**
3. Repeat this for all folds so that you get predictions for the entire training set, but none of those predictions were made on data the model was trained on
4. **These out-of-fold predictions become the training data for the meta-model**

Stacking - Data leakage Solution: Out-of-Fold Predictions

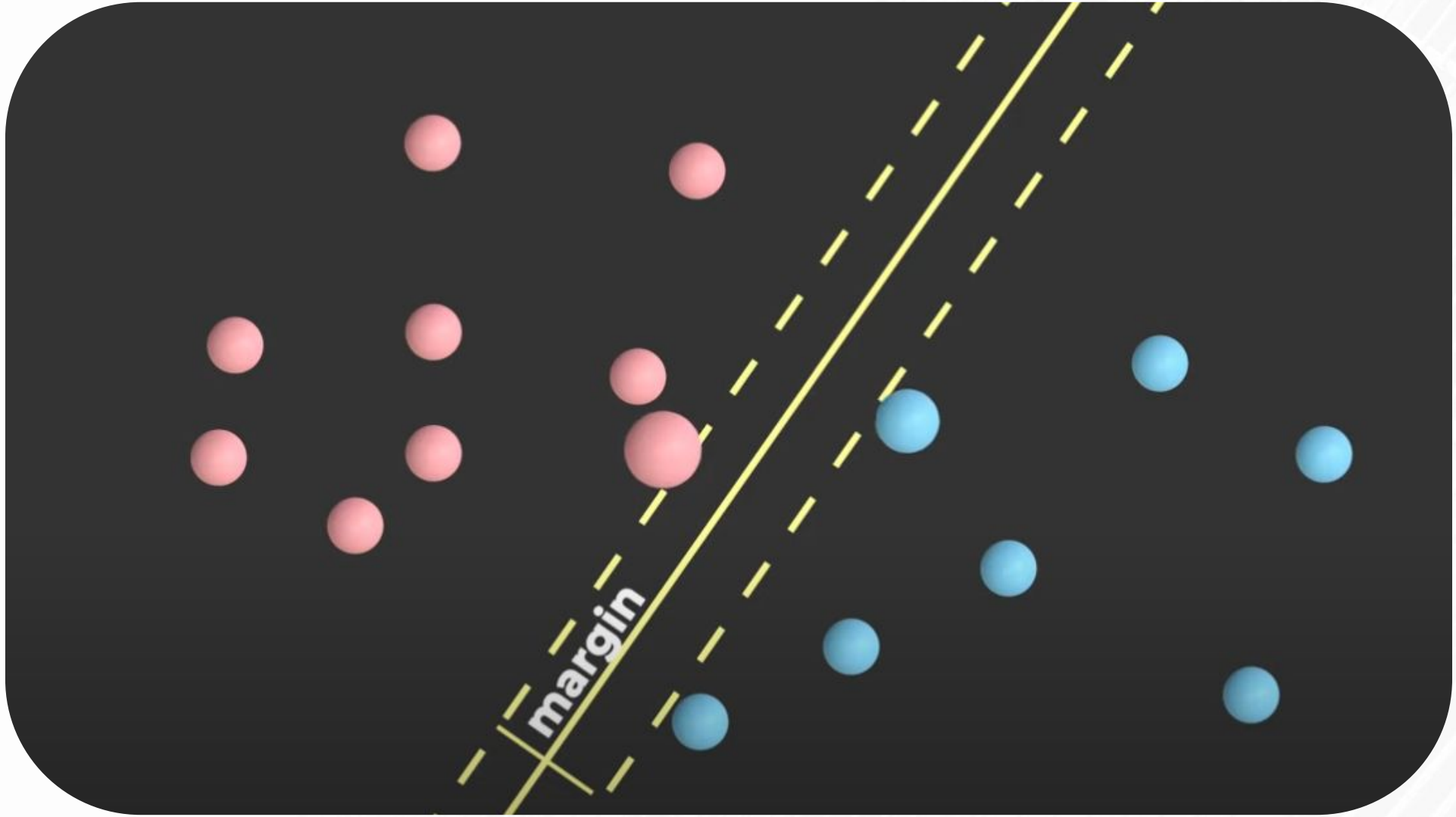
🔧 Example with 5-fold CV:

Fold	Base Model Trained On	Meta-Model Input Predictions Made On
1	Folds 2–5	Fold 1
2	Folds 1,3–5	Fold 2
…	…	…
5	Folds 1–4	Fold 5

In the end, you get predictions for the entire training set, but none of them were made on data the base model had seen.



Support Vector Machine (SVM)


margin

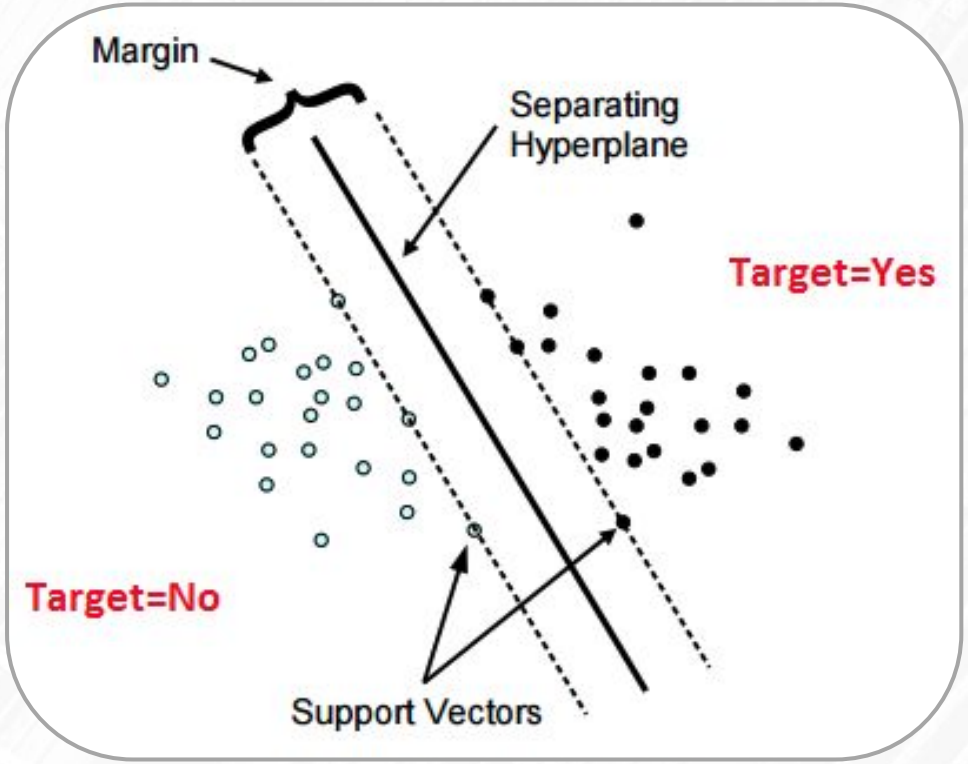
SVM - Algorithm Overview & Properties

- **Supervised** Learning
- **Eager** Learner - builds the model during training, makes predictions instantly during inference
- **Non-Parametric** model - the complexity of the model depends on the data itself, especially on the number and location of **support vectors**
- Suitable for **Classification** and **Regression** tasks, but is **primarily used for classification** tasks
- Linear SVMs (with a linear kernel) are fairly interpretable, while non-linear kernels (like RBF, polynomial), interpretation becomes much harder
- SVM relies on distances, hence **feature scaling is very important for SVM**

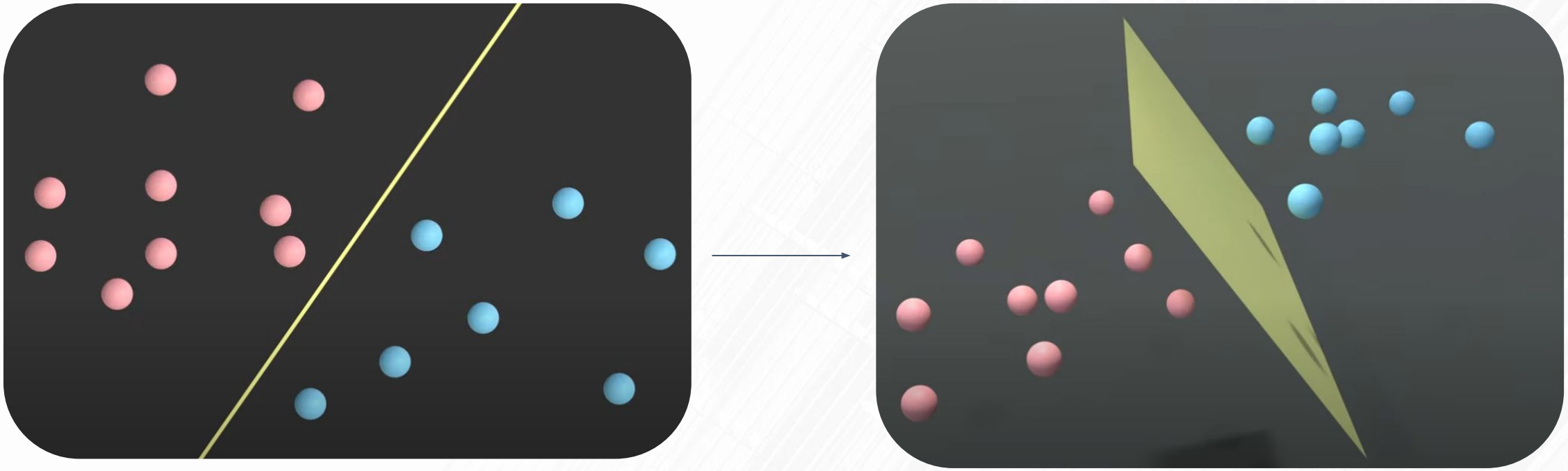
JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

SVM Terminology

- **Hyperplane** - a line (2D), plane (3D), or hyperplane (higher dimensions) that separates classes.
- **Margin** - distance between the hyperplane and the closest points from both classes (support vectors). SVM finds the maximum margin hyperplane.
- **Support Vectors** are the data points closest to the hyperplane. These points are critical — they define the decision boundary. Moving them changes the hyperplane; others don't.

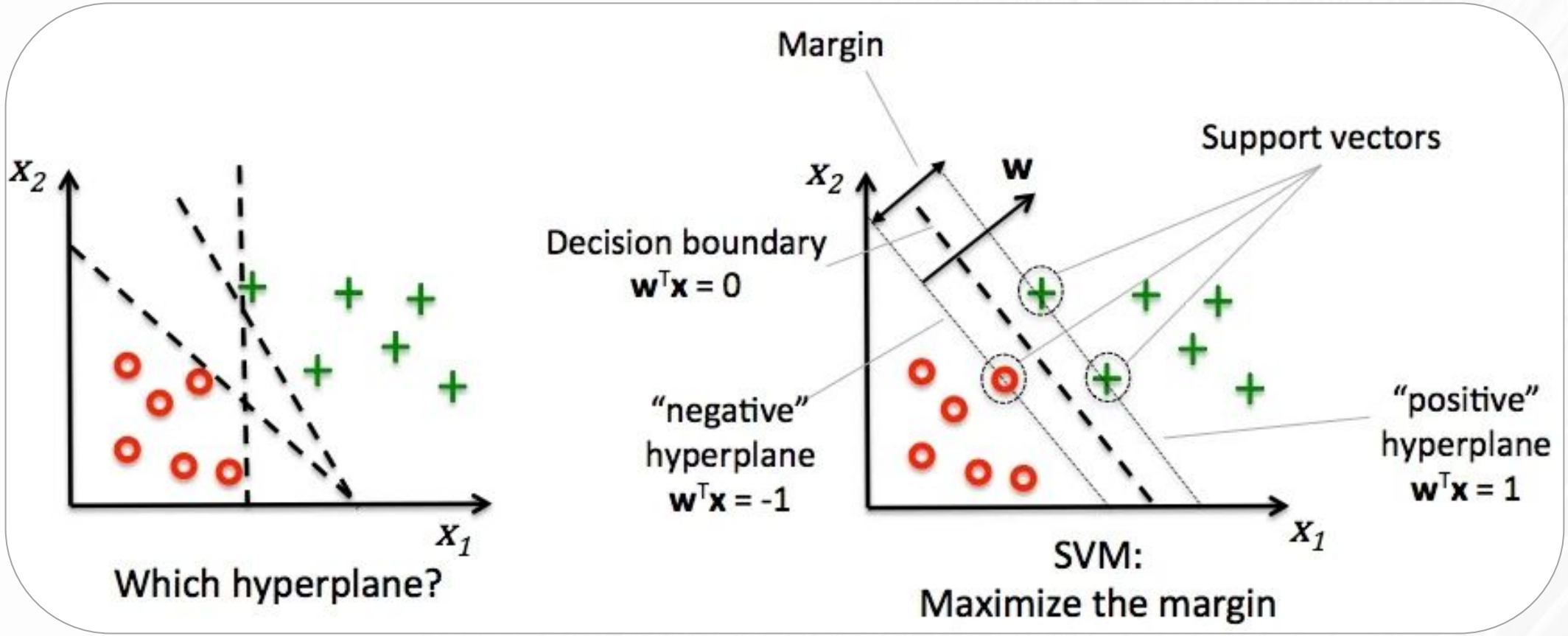


Linearly Separable Case in 2D (line), 3D (plane), 4D+ (hyperplane)



JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

Maximum margin hyperplane



JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

SVM Optimization Problem (Hard Margin)

Objective:

- Find the **hyperplane** that separates the two classes with the **largest possible margin**

While ensuring that (constraint):

- All points from Class A lie on one side of the margin
- All points from Class B lie on the opposite side
- The points that are closest to the hyperplane (called support vectors) lie exactly on the margin boundaries, helping to define the optimal hyperplane

Objective:

$$\text{Minimize} \quad \frac{1}{2}\|w\|^2$$

While ensuring that (constraint):

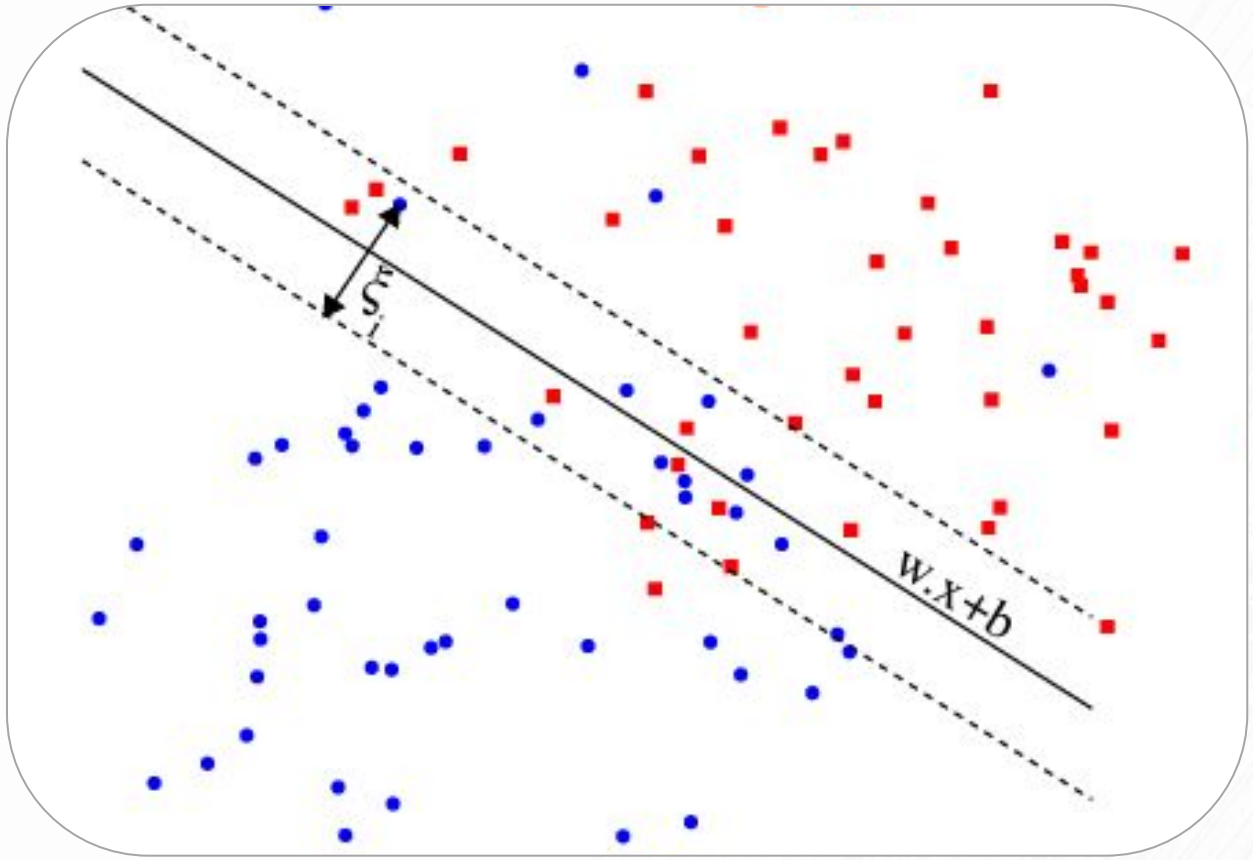
$$y_i(w \cdot x_i + b) \geq 1 \quad \text{for all } i$$

where:

- $y_i \in \{-1, 1\}$ are the class labels
- x_i are the feature vectors
- b is the bias (intercept)

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

SVM Optimization Problem (Soft Margin)



SVM Optimization Problem (Soft Margin)

Objective:

- Find the hyperplane that separates the two classes with the **largest possible margin**, while **allowing some points to slightly violate the margin** if necessary.

While ensuring that (constraint):

- **Most** points from Class A should lie on one side of the margin.
- **Most** points from Class B should lie on the opposite side.
- The points closest to the hyperplane (**support vectors) either lie exactly on the margin boundaries or may slightly cross it**, helping to define the optimal hyperplane.

Objective:

$$\text{Minimize} \quad \frac{1}{2}\|w\|^2 + C\sum_i \xi_i$$

While ensuring that (constraint):

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Where:

- ***C*** is a regularization parameter (trade-off between maximizing margin and minimizing classification error)
- ξ_i - slack variable, measures how much the i-th sample violates the margin

Which Points Matter in SVM?

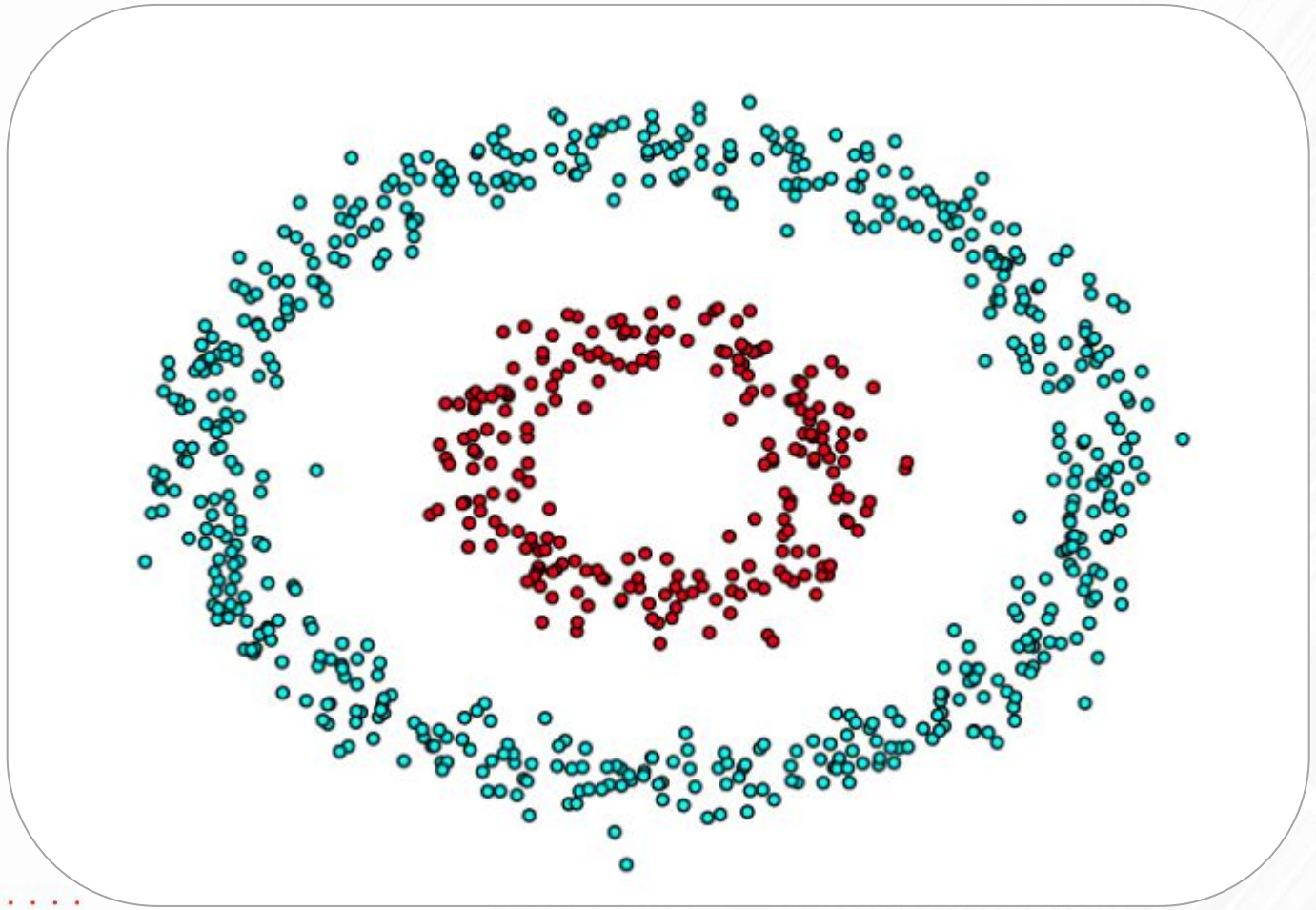
During training, SVM considers **all points** to set up the optimization problem.

- Every point must satisfy the margin constraints (with slack if needed).
- The total loss depends on all points (especially in soft-margin SVM).

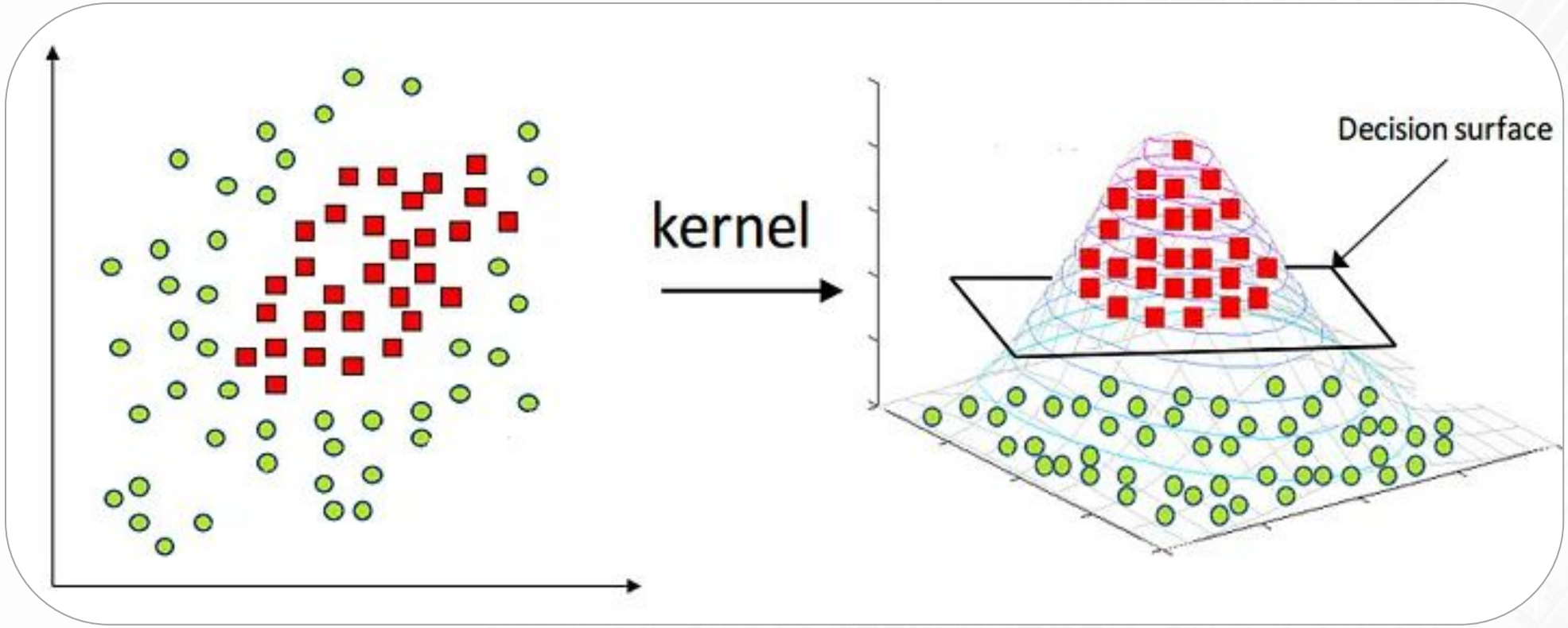
After training, only the **support vectors define the decision boundary**.

- Support vectors are the critical points that lie on or inside the margin.
- Non-support vectors have no influence on the final hyperplane.

What if the data is **not** Linearly separable???



SVM Kernels



SVM Kernel - Main Idea

- Real-world data is often not linearly separable in the original feature space.
- A simple straight line (or hyperplane) cannot perfectly separate the classes.

Solution:

- Map the original features into a higher-dimensional space, where the data becomes linearly separable.
- In other words, map each data point x to a higher-dimensional space using a transformation $\phi(x)$

But wait, there is a problem:

- This mapping could massively increase the number of features — sometimes to thousands, millions, or even infinite dimensions!
- Storing, computing, and optimizing over such a large feature space would be very expensive in terms of memory, compute power, algorithm complexity

Kernel Trick



JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

Prephase - Primal vs Dual Formulation in SVM

Primal Form - we directly optimize the hyperplane parameters w and b

$$\text{Minimize} \quad \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$$

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i \quad \text{and} \quad \xi_i \geq 0 \quad \text{for all } i$$

- optimize weights on all features directly

Dual Form: we re-express the problem in terms of Lagrange multipliers αi (one per training example).

$$\text{Maximize} \quad \sum_{i=1}^{n}\alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\alpha_i\alpha_j y_i y_j (x_i \cdot x_j)$$

$$0 \leq \alpha_i \leq C \quad \text{for all } i \quad \text{and} \quad \sum_{i=1}^{n}\alpha_i y_i = 0$$

- we optimize relationships between data points represented by dot product

How the Kernel Trick Works

Problem:

- To separate data that isn't linearly separable, we would need to **explicitly map** each point x_i into a **higher-dimensional space** $\phi(x_i)$
- Then we would compute **dot products** $\phi(x_i) \cdot \phi(x_j)$ of these mappings in that space.

Challenge:

- **Explicitly mapping** data to higher dimensions is **computationally expensive** or even **impossible** (e.g., infinite dimensions).

Kernel Trick Solution:

- **Instead of computing** $\phi(x_i)$ **and** $\phi(x_j)$ **separately and then taking their dot product,**
- **We directly compute** $K(x_i, x_j)$**,** where K is a **kernel function** that **implicitly** represents $\phi(x_i) \cdot \phi(x_j)$

Result:

- No need to ever compute or store the higher-dimensional vectors!
- Computation stays efficient, even when working with very complex feature spaces.

Available Kernels

1. Linear Kernel

$$K(x_i, x_j) = x_i \cdot x_j$$

- No mapping — works in the original feature space.
- Best when data is already approximately linearly separable.

2. Polynomial Kernel

$$K(x_i, x_j) = (x_i \cdot x_j + c)^d$$

- Maps the data into a higher-degree polynomial space.
- d = degree of the polynomial, c = free parameter (typically c≥0)

3. Radial Basis Function (RBF) / Gaussian Kernel

$$K(x_i, x_j) = \exp\left(-\gamma \|x_i - x_j\|^2\right)$$

- Maps data into an infinite-dimensional space.
- Very flexible — can create highly nonlinear boundaries.
- γ controls how far the influence of a single point reaches

4. Sigmoid Kernel

$$K(x_i, x_j) = \tanh(\kappa(x_i \cdot x_j) + c)$$

- Similar to a neural network activation.
- Less common in practice for SVMs.

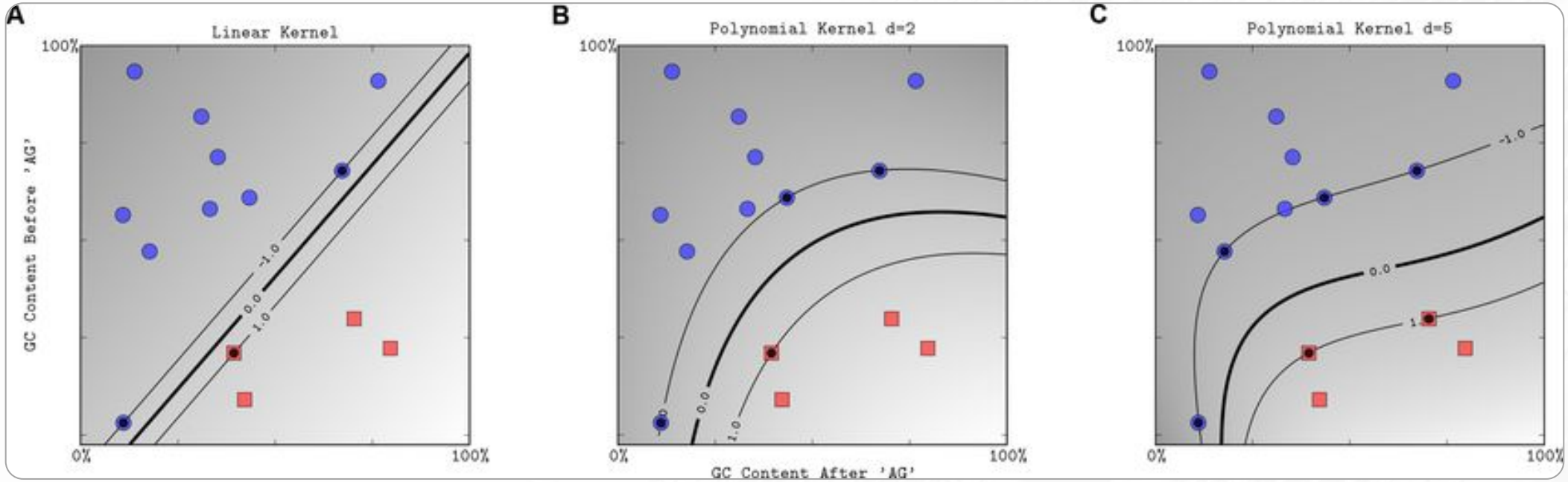
Kernel trick and decision boundary

- In most cases, the kernel trick makes the decision boundary nonlinear in the original feature space.
- In the original feature space (your raw features), SVM with a linear kernel creates a straight hyperplane → decision boundary is linear.
- When you apply a kernel (like polynomial or RBF kernel), you implicitly lift the data into a higher-dimensional space, where a simple linear separator (hyperplane) exists.
- However, when you map that separator back down to the original feature space,
- the boundary looks curved or nonlinear.

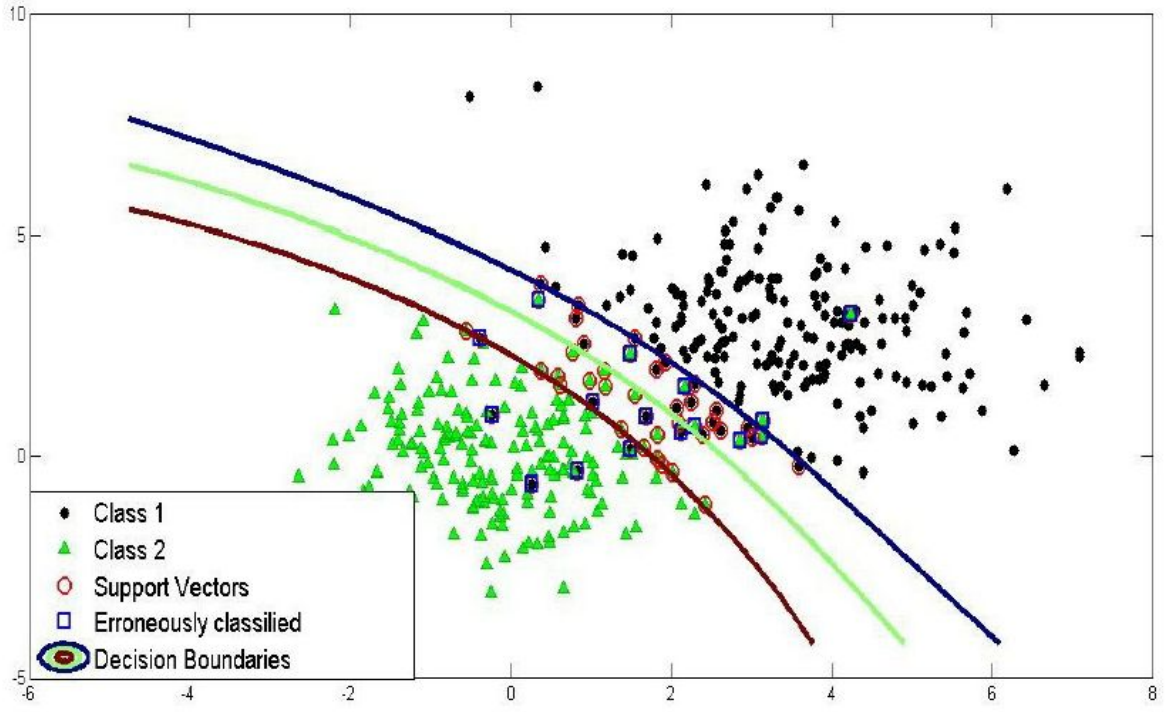
🪄 **That's the magic:**

- **In high-dimensional space → linear boundary.**
- **In original space → boundary appears nonlinear.**

Polynomial Kernel



RBF Kernel



Radial basis function kernel is a general purpose kernel

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

SVM vs Logistic Regression Decision Boundary

- SVM only cares about the points closest to the boundary (support vectors).
- It tries to maximize the margin — the distance between the closest points of different classes and the hyperplane.
- SVM optimization goal:

  maximize margin between classes

- Logistic Regression cares about all points — not just those near the boundary.
- It tries to find a boundary that best separates the classes by minimizing a probability-based loss function (the log loss or cross-entropy).
- Logistic Regression optimization goal:

  minimize classification error (log-loss) across all points

SVM vs Logistic Regression Decision Boundary

SVM:

➔ "I care most about the hardest-to-classify points (near the margin) and push them away as much as possible."

Logistic Regression:

➔ "I care about making the probability correct for every point, even ones that are far away from the boundary."

SVM vs Logistic Regression

When to prefer SVM:

- **You care more about classification (hard yes/no) than probabilities**
- Clear margin of separation exists between classes (even if messy).
- You expect high-dimensional data (SVM handles that well).
- Dataset is small to medium size (SVM training scales worse for huge datasets).
- You expect outliers — and you want SVM's margin to be robust.

When to prefer Logistic Regression:

- **You need probabilistic outputs (e.g., "80% chance of fraud")**
- The dataset is large — logistic regression scales much better to massive data.
- Relationships are approximately linear between features and the log-odds of the target.
- You want a simple, easily interpretable model (feature importance easy to read).
- You don't expect a very clear "gap" between classes (overlapping classes).

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

SVM Hyperparameters - C

1. **Regularization Parameter C**
   - Controls the **trade-off between maximizing the margin** and **minimizing classification error**.
   - **Small C:**
     - Softer margin
     - Allows more misclassifications
     - More generalized, less overfitting
   - **Large C:**
     - Harder margin
     - Tries to classify all training examples correctly
     - Risk of overfitting

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

SVM Hyperparameters - C

C Size	Regularization Strength	Behavior
Small C	**Strong regularization**	Wide margin, allow mistakes
Large C	**Weak regularization**	Tight margin, fewer mistakes

SVM Hyperparameters - Choice of Kernel

- Linear, Polynomial, RBF, Sigmoid.
- Different kernels can dramatically change the decision boundary.

SVM Hyperparameters - RBF Kernel Parameters

γ (Gamma)

- Controls the **influence** of a single training example.
- **Small gamma:**
  - Larger influence zone
  - Smoother decision boundary

- **Large gamma:**
  - Focuses on closer points
  - Very complex decision boundary (risk of overfitting)

JOHN BRYCE
תלמדו הייטק. זה עובד!
a matrix company

SVM Hyperparameters - Polynomial Kernel Parameters

- Degree d - Controls the complexity of the polynomial curve.
- Coefficient c - Controls the impact of higher-order terms.

SVM with multi-class

- SVM is originally a binary classifier
- So if we want to solve multi-class problem (2 classes or more), we will use one of multi-model strategies:
  - One-vs-Rest
  - One-vs-One

SVR (Support Vector Regressor)

- SVR applies the principles of SVM for regression tasks instead of classification.
- It tries to fit a function that deviates from the actual target values by no more than ε (a predefined margin).
- How SVR Works:
  - Define a margin of tolerance (ε) — errors within ε are ignored.
  - Only points outside the ε-tube (the margin) contribute to the model (become "support vectors").

